SYSTEM DOCUMENTATION

PoiseAtHome

Home-salon service booking and operations platform for Poise Hair Center Salon.

Architecture: React + Tailwind CSS · Laravel REST API · MySQL 8
Infrastructure: Docker Compose
Document date: 22 August 2026

1. System Overview

PoiseAtHome enables customers to discover salon services, book an eligible hairstylist for a home appointment, submit a downpayment for manual verification, and follow the appointment through completion. Staff and administrators use role-specific workspaces to operate schedules, services, payments, inventory, notifications, and reports.

Primary objectives

- Provide a clear customer journey from service discovery to completed appointment.
- Protect schedules, payment proofs, inventory quantities, and role-restricted data.
- Preserve historical appointment details with backend-generated snapshots.
- Give staff operational tools without exposing administrator functions.
- Give administrators practical oversight using real transactional data.

In scope

Authentication, email verification, password recovery, service catalog, staff qualification and availability, booking, manual payments, inventory, notifications, and reporting.

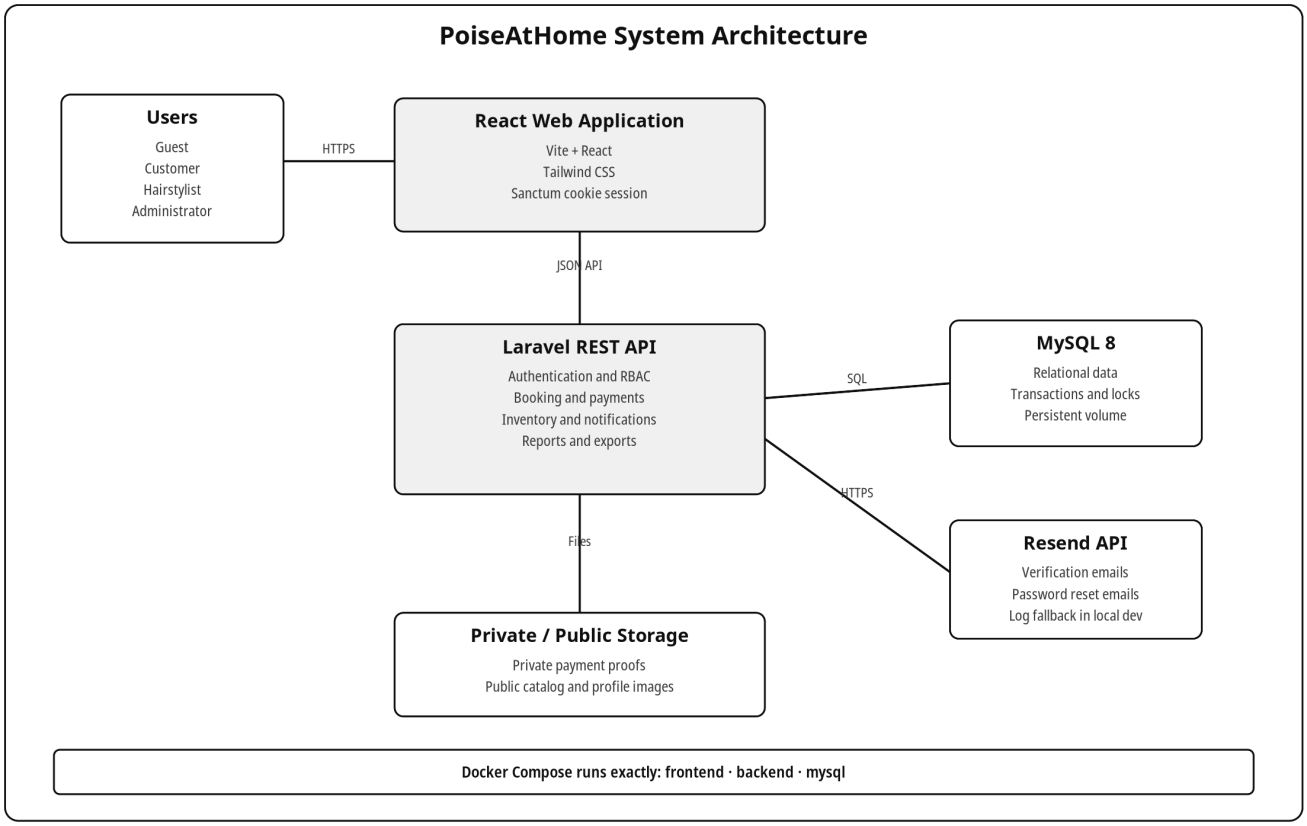
System boundary

The application integrates with Resend for transactional authentication email. It does not include a payment gateway, SMS, maps, payroll, chat, or other unrelated subsystems.

Stakeholders

Role	Goal
Customer	Book and track a home salon appointment securely.
Hairstylist	Manage availability and carry out assigned appointments.
Administrator	Manage salon operations, review payments, inventory, and reports.

2. Architecture

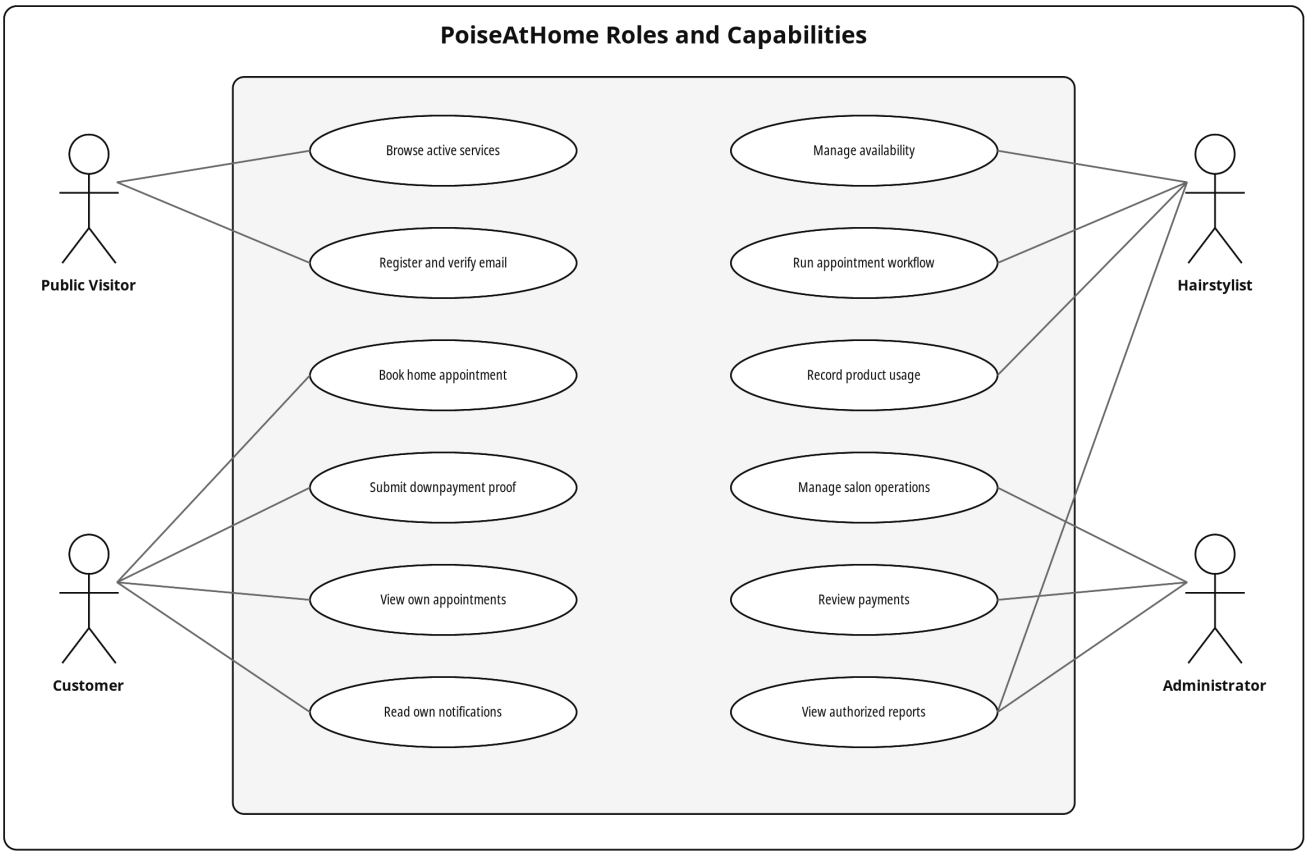


> The browser communicates with the Laravel API using Sanctum SPA authentication. Laravel remains the authority for validation, RBAC, calculations, state changes, and file authorization.

Runtime services

Service	Responsibility
frontend	React SPA, routing, accessible interface, and API interaction.
backend	Laravel REST endpoints, business rules, transactions, files, and exports.
mysql	Persistent relational data with keys, indexes, and unique constraints.

3. Roles, Authentication, and Security



Authentication controls

- Sanctum cookie-based SPA sessions and CSRF protection.
- Public registration always creates a customer account.
- Six-digit verification and password-reset codes are stored as hashes, expire, and are single use.
- Resend delivers transactional authentication email when configured; local development can use the controlled log fallback.
- Inactive accounts are not reactivated by password recovery.

Authorization controls

Route middleware, controller scoping, and ownership checks separate customer, staff, and administrator data. Private payment proofs are served only through authorized endpoints.

4. Functional Modules

Module	Implemented responsibility
Services	Categories, active catalog, price, duration, images, home-service flag, and staff qualification.
Appointments	Availability, notice and advance rules, days off, overlaps, snapshots, assignment, cancellation, and lifecycle.
Payments	Configured downpayment, manual submission, private proof, administrator verification or rejection, and confirmation guard.
Inventory	Products, decimal stock, immutable movements, appointment usage, safe reversal, and stock alerts.
Notifications	Per-user persisted messages, unread state, role-aware references, and alert deduplication.
Reports	Authorized operational summaries, completed service value, verified payment value, inventory, usage, CSV, PDF, and print.

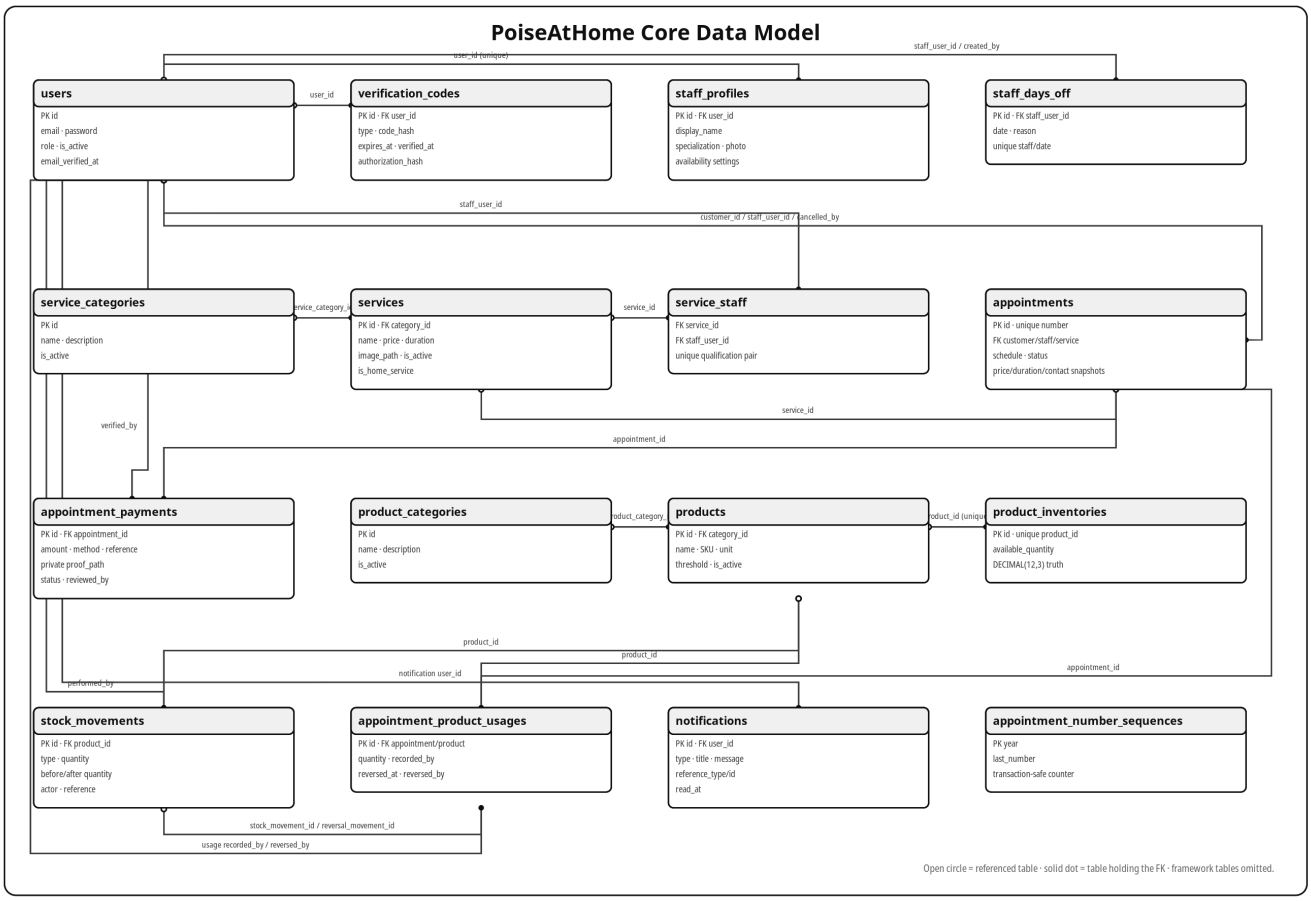
Appointment state

pending → **confirmed** → **in progress** → **completed**. Cancellation follows role and state rules. Completed and cancelled records cannot be returned to an earlier active state.

Financial terminology

Completed Service Value is based only on completed appointment price snapshots. Verified Payment Value includes only verified payment amounts. Neither metric is presented as profit or accounting revenue.

5. Core Data Model

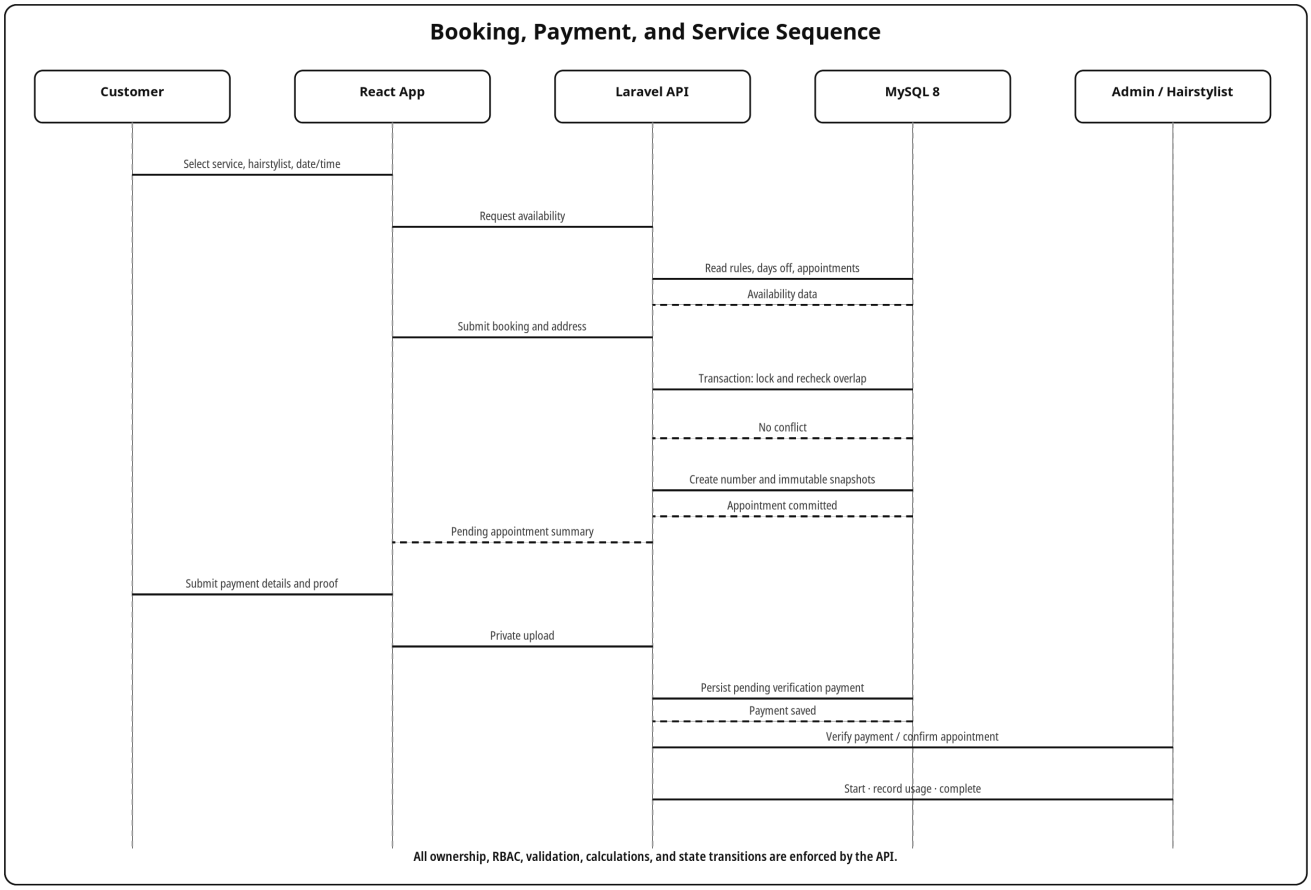


> The diagram intentionally focuses on business entities. Laravel framework tables for sessions, cache, jobs, and legacy password tokens are not shown.

Data integrity highlights

- Appointment numbers have a database unique constraint and a transaction-safe yearly sequence.
- Appointment snapshots retain the service, price, duration, customer name, and contact values at booking time.
- `product_inventories.available_quantity` is the current stock source of truth.
- Stock movements remain an immutable ledger; corrections use compensating movements.
- Payment proof paths are private implementation details and are not exposed in ordinary API resources or exports.

6. Booking and Payment Sequence



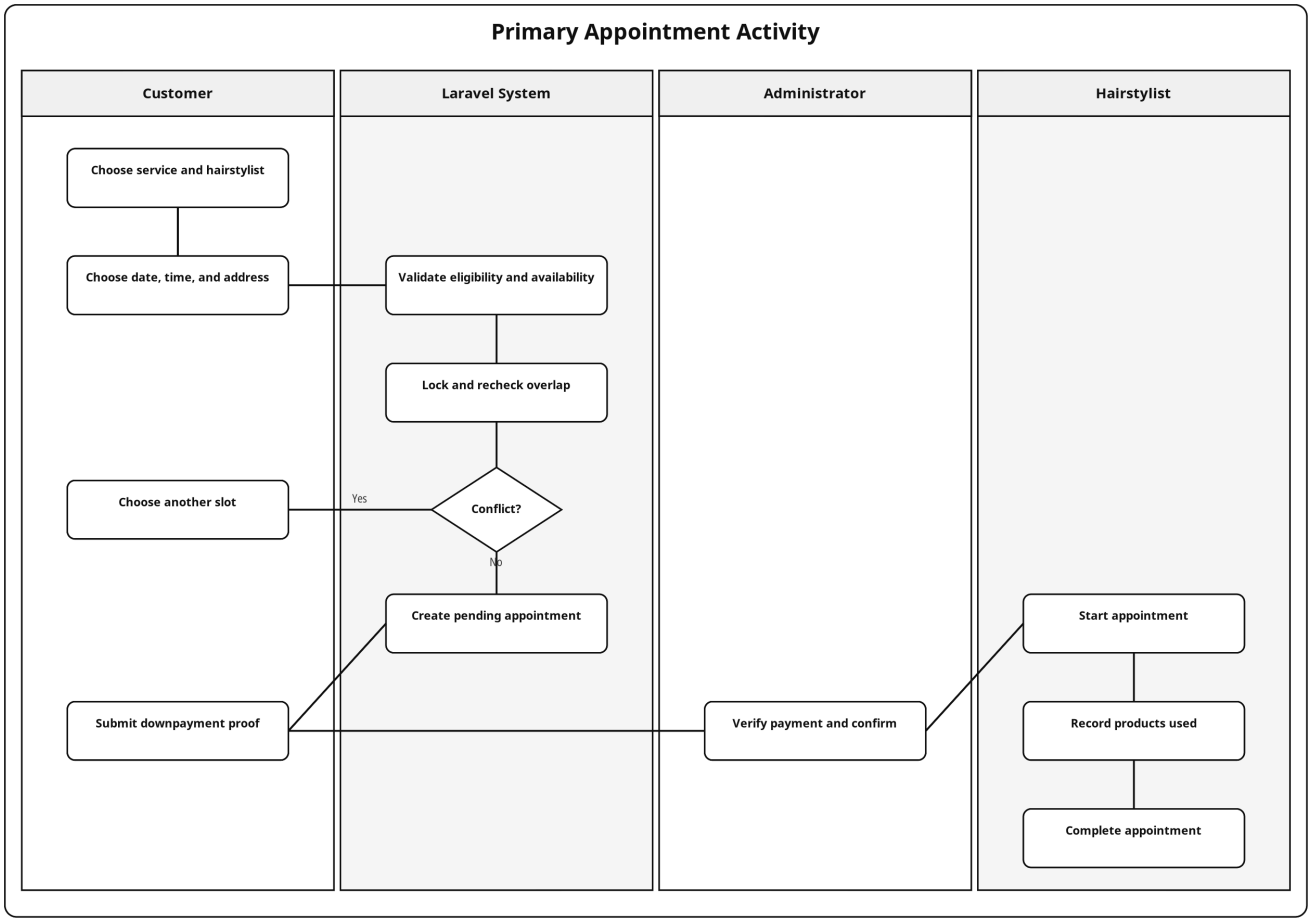
Scheduling safety

Booking and reassignment validate service activity, staff activity and qualification, availability, days off, and overlap. Overlap uses the rule `existing_start < requested_end AND existing_end > requested_start`. The overlap check is repeated inside the transaction after locking.

Payment confirmation

The required downpayment is calculated from the appointment price snapshot and configured percentage. Confirmation is rejected until verified payment value reaches the required amount.

7. Primary Activity



Operational outcome

A successful booking creates a pending appointment and reserves an eligible slot. Manual payment review enables confirmation. The assigned hairstylist starts the visit, records product consumption when applicable, and completes the appointment. Notifications are created after successful transactions so delivery problems do not roll back critical operations.

8. API and Integration Summary

Area	Representative route family	Access
Health and catalog	`/api/v1/health`, services	Public read
Authentication	`/api/v1/auth/*`	Guest or authenticated by operation
Customer	appointments, payments, notifications	Authenticated owner
Staff	`/api/v1/staff/*`	Authenticated staff, self-scoped
Administration	`/api/v1/admin/*`	Authenticated administrator
Reports and exports	report JSON, CSV, PDF	Same role policy for every format

Resend integration

Laravel sends verification and password-reset messages through the Resend HTTP API using server-only configuration. Provider failures are logged safely without exposing API credentials or provider internals to the client.

Files

Catalog and profile imagery may be public. Payment proofs remain private and require an ownership or administrator authorization check for access.

9. Deployment and Configuration

Local startup

1. Copy root and backend example environment files where required.
2. Set database and application values without committing secrets.
3. Run `docker compose up -d --build`.
4. Run `docker compose exec backend php artisan migrate --force`.
5. Run `docker compose exec backend php artisan db:seed` for intended development data.

Key environment groups

Group	Examples
Application	`APP_ENV`, `APP_URL`, frontend URL
Database	`DB_HOST`, database name, user, password
Sanctum/CORS	stateful domains and allowed origins
Email	`RESEND_API_KEY`, sender email, sender name
Business	downpayment, operating hours, booking constraints

> Exactly three Docker Compose services remain: mysql, backend, and frontend. Resend is an external HTTP integration, not another container.

10. Verification and Maintenance

Routine checks

- `docker compose exec backend php artisan test`
- Frontend production build in the canonical frontend environment.
- `docker compose config --quiet`
- `GET /api/v1/health` reports success and a connected database.

Documentation maintenance

Run `node docs/generate-docs.js` after material architecture, workflow, or schema changes. Editable SVG and HTML sources are retained beside the generated PNG, PDF, and DOCX outputs.

Known boundaries

Email delivery requires valid Resend server-side configuration in production. Payment verification remains manual. The application intentionally does not include online payment processing, SMS, maps, real-time sockets, payroll, ratings, or other unrelated features.

Document conventions

All diagrams use a monochrome visual language: white surfaces, black text and borders, and neutral gray grouping. No colored fills or brand accents are used in the technical diagrams.